

Grado: Ingeniería Electrónica de Comunicaciones
Asignatura: Procesamiento de Señales
Curso: 2025/2026

Práctica 6
Variables aleatorias

Objetivos

El objetivo de esta práctica es:

- Utilizar el método de la ruleta para generar muestras de una variable aleatoria.
- Caracterizar comportamiento de diferentes variables aleatorias.

Generación de muestras de una Variable Aleatoria

La generación de muestras de una variable aleatoria es importante para muchas aplicaciones prácticas, por ejemplo para simular procesos aleatorios que describen un fenómeno a través de una distribución de probabilidad, o para resolver integrales de las que no se conoce una solución cerrada. Generalmente se requiere generar muestras (números aleatorios) de forma que la frecuencia de ocurrencia se corresponda con una distribución de probabilidad que describe el comportamiento de la variable aleatoria.

En la práctica, los computadores no son capaces de generar números realmente aleatorios, en su lugar utilizan lo que se denomina números pseudoaleatorios. Estos son números que se generan de forma procedimental (es decir, a través de un algoritmo determinista) pero desde un punto de vista estadístico es posible tratarlos como si fueran aleatorios.

Un algoritmo sencillo y muy utilizado para generar números pseudoaleatorios es el generador lineal congruente, cuyo funcionamiento se basa en generar una secuencia de números a través de una relación de recurrencia de la forma $X(n+1) = (aX_n + c) \bmod m$. Según la elección particular de los parámetros a, c y m, la secuencia generada presentará un comportamiento más o menos adecuado como secuencia aleatoria. Es posible realizar diferentes pruebas que verifican si el algoritmo genera números con las propiedades estadísticas deseadas y así poder tratarlos como aleatorios.

Generación de Variables Aleatorias en Matlab

Matlab proporciona diferentes funciones para generar muestras de variables aleatorias con las distribuciones de probabilidad más comunes:

- `randi(N,P,Q)` - Genera una matriz de tamaño PxQ con muestras de una variable discreta con N casos equiprobables: $p(x_i) = \frac{1}{N}, i = 0, 1, ..., N-1$.
- `rand(P,Q)` - Genera una matriz de tamaño PxQ con muestras de una variable discreta con una distribución uniforme en el intervalo $[0, 1]$: $p(x) = \mathcal{U}_{[0,1]}(x)$. A partir de esta se puede generar cualquier distribución uniforme. Dada $X \sim p_X(x) = \mathcal{U}_{[0,1]}(x)$, podemos construir una distribución uniforme arbitraria mediante $Y = a + (b-a)X$. Se demuestra trivialmente que $p_Y(x) \sim U_{[a,b]}(x)$.
- `randn(P,Q)` - Genera una matriz de tamaño PxQ con muestras de una variable discreta con una distribución gaussiana de media $\mu = 0$ y varianza $\sigma^2 = 1$: $p(x) = \mathcal{N}_{(0,1)}(x)$. Al igual que en el caso anterior, si $X \sim p_X(x) = \mathcal{N}_{[0,1]}(x)$, y hacemos $Y = \mu + \sigma X$, es fácil ver que $p_Y(x) \sim \mathcal{N}_{[\mu,\sigma]}(x)$.

Para generar otras funciones de probabilidad, debemos recurrir a los algoritmos que hemos visto en las clases teóricas, que presentamos brevemente a continuación.

Método de la ruleta

Este método se inspira en el conocido juego de azar[1]. Primero se divide la ruleta en diferentes sectores, proporcionales a las probabilidades de la función que se desea muestrear. A continuación se simula el lanzamiento, y se genera la muestra según en que sector caiga la bola virtual. Para implementar en Matlab, la ruleta se hace corresponder con el intervalo $[0, 1]$, que se divide en tramos proporcionales a las probabilidades que se quiere muestrear, y el lanzamiento de la bola se simula mediante el muestreo de una v.a. uniforme en el mismo intervalo, $V \sim U_{[0,1]}(x)$. El código se lista a continuación:

```
function x = ruleta(px, N)
  Px = cumsum(px);
  v = rand(N,1);
  x = sum((v > Px)') + 1;
end
```

[1] Por si hay algún despistado, el juego consiste en lanzar una bola dentro de una ruleta que gira, dividida en diferentes sectores. La bola puede caer en cualquiera de las divisiones. El juego en sí no tiene mucha gracia, así que para hacerlo más divertido se suelen apostar grandes cantidades de dinero.



Método de muestreo por rechazo

Este método parte de una idea parecida al de la ruleta. En esencia, se genera una muestra de un distribución conocida (p.ej. la uniforme) y se utiliza una ruleta para aceptar o rechazar la muestra, con una probabilidad determinada según la proporción entre la distribución objetivo y la auxiliar.

```
function x_aceptadas = muestreo_rechazo(N)
  M = 2;
  x_aceptadas = [];
  N_aceptadas = 0;
  while N_aceptadas < N
    x_q = rand(N - N_aceptadas, 1);
    y = rand(N - N_aceptadas, 1);
    I = find(y*M.*q(x_q) - p(x_q) < 0); % Muestras aceptadas
    x_aceptadas = [x_aceptadas; x_q(I)];
    N_aceptadas = length(x_aceptadas);
  end
end

function y = p(x)
  % Densidad probabilidad objetivo
  y = ...
end

function y = q(x)
  % Densidad probabilidad auxiliar
  y = 1
end
```

Muestreo de una mezcla

Este método se utiliza para generar muestras de una v.a. con una densidad de probabilidad de la forma $p(x) = \sum_{i=1}^{N} p(x|x_i)p(x_i)$, es decir, una mezcla de varias conocidas. La idea es utilizar una ruleta para decidir a cual de los grupos pertenece la muestra y a continuación utilizar el método que corresponda según la distribucion en particular. A continuación se lista el pseudocódigo del algoritmo:

```
N = 1000;
px = [0.4, 0.2, 0.4];
for i = 1:N
  tipo = ruleta(px, 1);
  x(i) = generar_muestra(tipo);
end

function x = generar_muestra(tipo)
  switch tipo,
    case 1, x = ...; break;
    ...
    case n, x = ...; break;
  end
end
```

Histograma

El histograma es una representación en forma de diagrama de barras de las frecuencias de ocurrencia de los valores de una v.a. Para variables discretas, el histograma es simplemente un recuento de las muestras. Si la variable es continua, se agrupan los valores por intervalos, y se realiza el recuento de las muestras que caen en cada intervalo. Si normalizamos el histograma para que su area se 1, lo podemos interpretar directamente como una aproximación de la densidad de probabilidad de la v.a. La normalización se consigue dividiendo por N, el número de muestras, si la v.a. es discreta. Si la variable es continua, hay que dividir por $N\Delta x$, donde Δx es la anchura del intervalo utilizado para el recuento.

Para calcular el histograma en Matlab tenemos la función `histogram`. Si ignoramos los valores devueltos, esta función muestra una gráfica con el histograma generado.

```
h = histogram(x);
```

Por el contrario, si recogemos el valor devuelto en una variable, la gráfica no se mostrará y a cambio tendremos disponible un objeto con toda la información:

```
>> h = histogram(x);
  h =

  Histogram with properties:

            Data: [1x500 double]
          Values: [1x14 double]
         NumBins: 14
        BinEdges: [1x15 double]
        BinWidth: 0.0100
       BinLimits: [0.6000 0.7400]
   Normalization: 'count'
       FaceColor: 'auto'
       EdgeColor: [0 0 0]

  Show all properties
```

Podemos utilizar esta información para normalizar el histograma.

```
h_normalizado = h.Values / h.BinWidth / N;
```

Estadísticos

Es sencillo calcular los estadísticos muestrales en Matlab directamente a partir de las fórmulas:

```
% x contiene las muestras de la v.a.
N = length(x);
media = sum(x) / N;
vcm = (x * x') / N;
varianza = sum((x - media).^2) / N;
desviacion = sqrt(varianza); % sqrt(sum((x - media).^2) / N);
```

También es posible usar las funciones `mean`, `std` y `var`.

```
media = mean(x);
varianza = var(x, 1); % El 1 es para normalizar por N en lugar de (N-1)
desviacion = std(x, 1);
```

Conocida la distribución de probabilidad de la v.a., en algunos casos es posible caracterizarla por completo mediante de uno o varios estadísticos. Por ejemplo, una distribución normal se parametriza por la media y varianza, es decir, el conocimiento de estos dos estadísticos permite reconstruir por completo la función de densidad de probabilidad. En otros casos el cálculo de los estadísticos no será suficiente.



Generación y análisis de variables aleatorias

Tarea 1: **Muestreo de variables aleatorias.** El objetivo de esta tarea es generar diferentes conjuntos de muestras de una variable aleatoria discreta mediante el método de la ruleta.

- ▷ $p_X(x_i) = [0{,}1, 0{,}25, 0{,}15, 0{,}35, 0{,}1, 0{,}05]$.

Para comprobar que las muestras generadas siguen la función de probabilidad propuesta:

a) Genera tres conjuntos de muestras de diferente tamaño: $N = 100$, $N = 1000$, $N = 10000$.

b) Representa en una gráfica los valores de las muestras.

c) Representa, en otra gráfica, los histogramas normalizados y la función de probabilidad propuesta.

d) Compara los histogramas obtenidos con la función de probabilidad. Explica la influencia de N en el resultado.

e) Calcula su media, valor cuadrático medio y varianza. Indica si la media y varianza caracterizan la función de probabilidad.

Nota: Únicamente se pueden usar los generadores aleatorios de Matlab randi, randn y rand. Por lo tanto, se recomienda utilizar el método de muestreo más sencillo que, utilizando dichas funciones, genere las muestras propuestas.

Tarea 2: **Caracterización de conjuntos muestrales desconocidos**

El objetivo de esta tarea es caracterizar conjuntos muestrales de origen desconocido a partir de su histograma. Los conjuntos muestrales que hay que analizar se generarán utilizando la función `DatosTarea2`. Para cada conjunto muestral:

a) Representa el histograma normalizado.

b) Indica si la variable es continua o discreta, y cuál es su función de probabilidad original.

c) Calcula los estadísticos habituales (media, valor cuadrático medio y varianza).

d) ¿Los estadísticos sirven para parametrizar correctamente la función de probabilidad de las muestras?

